

Individual Assignment Algorithms

Boyer-Moore complexity analysis

Joel

May 25, 2026

Contents

1	Introduction	1
2	Brief Description of the Boyer-Moore Algorithm	2
3	Implementation	2
4	Experimentation	3
5	Results and Analysis	3
6	Conclusions	5
7	References	6

1 Introduction

When the internet contains more characters than you could count, it is a given that you would want a fast string matching algorithm to go with that. One of the more efficient ones out there is created by Robert S. Boyer and J Strother Moore and called Boyer-Moore for short. This paper is looking to analyze the efficiency of this algorithm and compare it to a brute force approach.

2 Brief Description of the Boyer-Moore Algorithm

The goal of the algorithm is to find where and if a certain pattern matches somewhere in a text by skipping many unnecessary comparisons. It does this by following 2 rules, the bad character rule, and the good suffix rule. The bad character rule says if a character in a text is part of the pattern then shift so that the pattern aligns with that character, otherwise shift the entire pattern length. The good suffix rule instead looks at the pattern to see if there are any suffixes that matches a sub-string of the pattern. And if it can not find any, it instead looks at if there is a suffix of that suffix that matches a sub-string of the pattern. And if it can not find that either it shifts the length of the pattern.

3 Implementation

This implementation was done in the programming language C by following the formal description of Thierry Lecroq in his paper [1] "A fast implementation of the good-suffix array for the Boyer-Moore string matching algorithm" for the good suffix tabel. The bad character tabel was implemented using the pseudo code given by Mohammed Sahli and Tetsuo Shibuya [2]. And the complete Boyer-Moore algorithm was made with the pseudo code from the original paper [3] "A Fast String Searching Algorithm".

This assignment is part of the course TAIK12 Algorithms. The assignment focuses on the implementation and analysis of some fundamental sorting algorithms, as well as planning and executing experiments to evaluate their performance.

The main goal of this assignment is to develop a deeper understanding of how different sorting algorithms operate in practice, and how their performance can relate to the theoretical time complexities. In order to achieve this, three sorting algorithms were chosen and implemented in the programming language C, and their behavior was studied through experiments.

Furthermore, the assignment aims to introduce different empirical evaluation techniques by measuring the number of basic operations performed by each algorithms depending on input conditions. The results are then analyzed and compared to the theoretical expectations, allowing for better understanding of algorithm efficiency.

4 Experimentation

The Boyer-Moore algorithm was tested inside Visual Studio Code by counting every comparison made, which is shown by the OP counter (comparison counter). In this implementation I chose to end the algorithm if it found the pattern, instead of going through the whole text and possibly finding multiple occurrences. And that is why in the user-interface you can choose where that pattern should be in the text. If this would not be implemented and the pattern was placed randomly in the text, then the results of the op counter would be just as random and a lot harder to compare.

It is worth mentioning that if the algorithm was made to count to the end of the text to find all (if any) pattern occurrences. The program would have the same OP count as if the pattern was placed at the same spot but for the algorithm that stops when it finds the pattern. That is why I have chosen to not compare text lengths for my implementation but instead where the pattern is placed in the text.

I have chosen to gather data on this algorithm based on my hypothesis around it, by seeing how well it performs when the text has no similar characters to the pattern, when the characters are random, and when the text contains only characters that are part of the pattern. This is to see how well the bad char table and good suffix table will perform on different characters. Apart from this, I also looked at the length of the pattern to see if that makes an impact on the OP count.

5 Results and Analysis

for a pattern in alphabetical order from 'a' to pattern size (size of 6 would give a pattern abcdef). And for a text of size 1024 that only contains letters that are not part of the pattern we get the results seen in fig. 1.

When testing the same pattern and text size for a text that has random characters, we get a very similar output. Seen in fig. 2. This makes me conclude that since the alphabet used in text can be 94 different characters, the text generation seldom generates a character that would be part of the pattern either way. And if it does, it many times just adds 1 comparison to the OP counter.

Seeing as both of these tests decreased in OP count the bigger the pattern was. You can draw the conclusion that for a good pattern on an average text the complexity should be around $\theta(m/n)$. Where m is the index where the pattern is found and n is pattern length. This is seen more clearly with switching place of pattern size and where pattern is found shown in fig. 3.

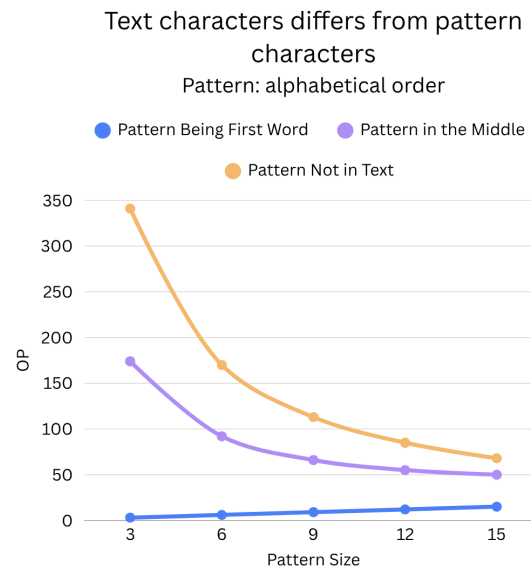


Figure 1: Fig. 1

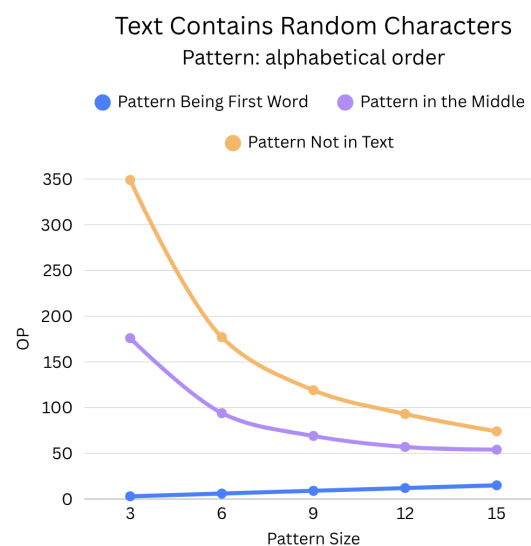


Figure 2: Fig. 2

For example we can see that for $m = 256$ and $n = 6$. You would theoretically get $256/6 \approx 46$. which is almost the same as comparing it to the empirical result of approximately 50 shown in fig. 3.

The multiplicative constant of this must then be based on the characters that are generated in the random text. Making the difference very small for large alphabets

and bigger for smaller alphabets.

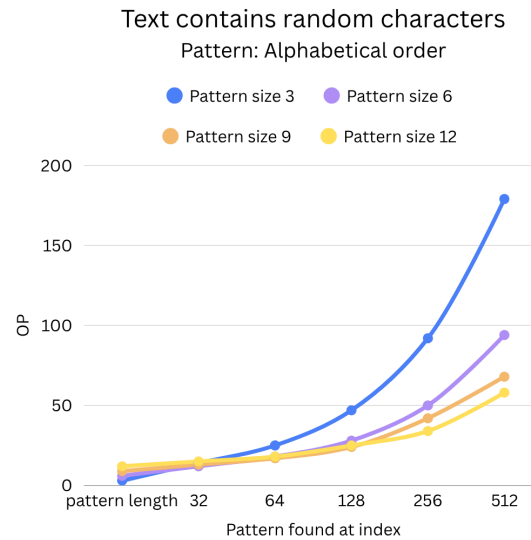


Figure 3: Fig. 3

6 Conclusions

In conclusion for these results is that the Boyer-Moore algorithm is complicated and the comparisons the algorithm has to make depends a lot on what type of text it is. Where the pattern is found, and how long the pattern is. So the worst case of this would be where both the good shift table and bad character table is completely useless and the pattern only shifts once every single time, just like a brute force approach, but still needs to check all the characters of the pattern. Concluding that the worst case is $O(m * n)$, where m is the length of the text and n is the length of the pattern.

seeing as the text length m can also be described as where the pattern is found (for the algorithm that stops when it finds the pattern) and the pattern is found at the first index of the text. That would give $m = 1$ and the best case scenario to be $\Omega(1 * n)$. Which is a constant and gives the complexity $\Omega(1)$.

What is more interesting is the average case which can be anywhere between n and nm . My conclusion is that it fully depends on how usefull the bad char table and good suffix table is. And how usefull these are depends on both the text and the pattern. For example I found that a text that has the same characters as the pattern has a very hard time using the different tables, and mostly shifts small

amounts at a time. While a text with no similar characters as the pattern can simply just shift past all of them by n steps each time. This type of text is however rarely the case, where normally the text contains more random characters. Making it again by chance.

The strengths and weaknesses of this algorithm was a lot more complex than I originally thought.

In conclusion, the experiments provided a clear comparison between the empirical results and the theoretical time complexities of the sorting algorithms. Overall, the results closely matched the expected behavior. Selection sort showed consistent quadratic growth regardless of input, confirming the $\theta(n^2)$ complexity in all cases. Insertion Sort varied depending on input, performing linearly for ordered data and quadratically for reverse and random inputs, demonstrating the sensitivity to its input order. Merge sort was the most efficient, with growth following $n * \log(n)$ and small dependence on input type.

7 References

@Book levitin11,

author = Anany Levitin,

title = Introduction to the Design and Analysis of Algorithms,

edition = Third,

year = 2011,

publisher = Pearson Education